

## Практическая работа: «Безопасный Todo-менеджер»

### Описание

Вы создаёте веб-приложение для управления задачами с полной системой безопасности: регистрация пользователей, роли, политики доступа, JWT для API и защита от атак.

**Стек:** ASP.NET Core, Razor Pages, EF Core, Identity, JWT.

---

### Подготовка

Создайте пустой ASP.NET Core проект. Подключите EF Core (SQLite или SQL Server — на выбор). Создайте базовую модель задачи:

- TaskItem: Id, Title, Description, CreatedAt, IsCompleted, UserId (внешний ключ).
- Связь: один пользователь — много задач.

Базовую CRUD-логику для задач можно сделать через Razor Pages — это не предмет оценки в этой работе. Фокус — на безопасности.

---

### Задание 1. Регистрация и вход через Identity

#### Что нужно реализовать

- Подключите ASP.NET Core Identity в проект. Используйте встроенный класс IdentityUser или создайте свой AppUser : IdentityUser с дополнительным полем FullName.
- Контекст базы данных должен наследоваться от IdentityDbContext.
- Настройте Identity с параметрами: минимальная длина пароля — 6 символов, требование цифры — обязательно, блокировка после 5 неудачных попыток на 15 минут.
- Создайте две Razor Pages: **Register** и **Login**. Формы — классические, с серверной валидацией и перезагрузкой.
- После успешной регистрации — автоматический вход и редирект на главную. После успешного логина — редирект на главную.
- Создайте страницу **Logout** (POST, не GET — чтобы по ссылке нельзя было разлогинить).
- На главной странице покажите имя пользователя, если он вошёл, и ссылки «Войти» / «Зарегистрироваться» — если нет.

#### Контрольная точка

- Регистрируюсь → автоматически вхожу → вижу своё имя на главной.
  - Выхожу → вижу «Войти» и «Зарегистрироваться».
  - Пытаюсь зарегистрироваться с паролем из 3 символов → ошибка валидации на сервере.
  - Пытаюсь войти с неверным паролем 5 раз → аккаунт блокируется на 15 минут.
- 

### Задание 2. Защита страниц и роли

#### Что нужно реализовать

- Создайте три роли при запуске приложения (seed): **Admin**, **Author**, **User**. Если роли уже существуют — не создавать заново.
- При регистрации новому пользователю автоматически назначается роль **User**.
- Создайте seed-пользователя-админа: при первом запуске в базу добавляется пользователь с логином admin@todo.com, паролем Admin123! и ролью **Admin**. Проверьте: при повторном запуске он не дублируется.
- Закройте страницу создания задачи атрибутом [Authorize] — только вошедшие могут создавать.
- Создайте страницу **Admin Panel** — доступ только для роли Admin ([Authorize(Roles = "Admin")]). На ней выведите список всех пользователей с их ролями.

- Обычный пользователь, пытающийся открыть Admin Panel, получает 403 Forbidden. Неавторизованный — редирект на логин.

### Контрольная точка

- Регистрируюсь как обычный пользователь → попадаю на главную, могу создать задачу.
  - Пытаюсь открыть /admin → 403 Forbidden.
  - Вхожу как admin@todo.com → открываю /admin → вижу список пользователей с ролями.
  - Пытаюсь открыть страницу создания задачи без входа → редирект на логин.
- 

## Задание 3. Политики и resource-based авторизация

### Что нужно реализовать

- Каждая задача привязана к пользователю, который её создал (TaskItem.UserId).
- Создайте политику **«CanEditTask»** — пользователь может редактировать и удалять только свои задачи. Админ может редактировать и удалять любые.
- Реализуйте кастомный AuthorizationHandler<CanEditTaskRequirement, TaskItem>, который проверяет: совпадает ли UserId задачи с ID текущего пользователя, или текущий пользователь — админ.
- Зарегистрируйте handler и политику в Program.cs.
- На странице редактирования задачи реализуйте resource-based авторизацию: загрузите задачу из базы, вызовите IAuthorizationService.AuthorizeAsync(User, task, "CanEditTask"). Если проверка не прошла — верните Forbid().
- На странице списка задач: рядом с каждой задачей показывайте кнопки «Редактировать» и «Удалить» **только если пользователь имеет право их видеть**. Чужие задачи — только просмотр. Реализуйте проверку через IAuthorizationService прямо в Razor Page.

### Контрольная точка

- Пользователь А создаёт задачу «Купить хлеб».
  - Пользователь В открывает список — видит задачу А, но без кнопок «Редактировать» и «Удалить».
  - Пользователь В пытается открыть URL /tasks/edit/1 (задача А) напрямую → 403 Forbidden.
  - Пользователь А открывает /tasks/edit/1 → видит форму редактирования.
  - Админ открывает /tasks/edit/1 → видит форму (может редактировать любую).
- 

## Задание 4. JWT для REST API

### Что нужно реализовать

- Добавьте пакет Microsoft.AspNetCore.Authentication.JwtBearer.
- Настройте две схемы аутентификации: Cookie (для Razor Pages) и JWT (для API). Укажите JWT как схему по умолчанию для API-контроллеров.
- Секретный ключ храните в appsettings.json (длина — не менее 32 символов).
- Создайте API-контроллер AuthApiController с эндпоинтом POST /api/auth/login. Он принимает e-mail и пароль, проверяет через SignInManager.CheckPasswordSignInAsync, и если верно — выпускает JWT с claims: ID пользователя, e-mail, роли. Срок жизни токена — 1 час.
- Создайте API-контроллер TasksApiController с CRUD для задач. Защитите его атрибутом [Authorize] (JWT). Без токена — 401.
- При создании задачи через API — UserId берётся из claims токена, а не из тела запроса. Пользователь не может указать чужой UserId.
- При получении списка задач через API — обычный пользователь видит только свои задачи. Админ — все.
- Подключите Swagger. Настройте Swagger так, чтобы можно было вставить JWT-токен в интерфейс и тестировать эндпоинты с авторизацией.

- Создайте .http-файл с запросами: логин (получить токен), получить задачи (с токеном), создать задачу (с токеном), получить задачи без токена (должен быть 401).

### Контрольная точка

- POST /api/auth/login с правильным паролем → получаю JWT.
- GET /api/tasks без токена → 401 Unauthorized.
- GET /api/tasks с токеном пользователя А → только задачи А.
- GET /api/tasks с токеном админа → все задачи.
- POST /api/tasks с токеном → задача создаётся с UserId из токена.
- В Swagger вставил токен → эндпоинты работают. Не вставил — 401.

---

## Задание 5. Защита от CSRF и XSS

### Что нужно реализовать

- Включите глобальный антифоггери-фильтр для всех POST-форм Razor Pages через `AutoValidateAntiforgeryTokenAttribute`.
- Убедитесь, что все формы на страницах (создание задачи, редактирование, логин, регистрация, logout) содержат антифоггери-токен. Razor Tag Helpers добавляют его автоматически — проверьте, что forms используют asp-\* helpers, а не чистый HTML.
- Создайте AJAX-действие: кнопка «Отметить выполненной» на задаче. При нажатии отправляется POST через fetch без перезагрузки.
- Реализуйте передачу антифоггери-токена в AJAX: токен кладётся в `<meta name="csrf-token">` на странице, JavaScript читает его и отправляет в заголовке X-CSRF-TOKEN. Сервер настраивается читать токен из этого заголовка.
- Проверьте: AJAX-запрос без заголовка X-CSRF-TOKEN → 400 Bad Request.
- Добавьте CSP-заголовок: скрипты только с 'self', инлайн-скрипты запрещены. Добавьте остальные security headers: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Strict-Transport-Security.
- Проверьте: если в описании задачи написать `<script>alert('XSS')</script>` и сохранить — при выводе на странице скрипт не выполняется (Razor кодирует автоматически). Если CSP включён — инлайн-скрипт блокируется даже при ошибке.

### Контрольная точка

- Отправляю форму создания задачи → работает (токен в форме есть).
- Нажимаю «Отметить выполненной» через AJAX → работает (токен в заголовке).
- Отправляю AJAX-запрос без токена → 400 Bad Request.
- Создаю задачу с заголовком `<script>alert(1)</script>` → сохраняется. Открываю список — alert не срабатывает, текст отображается как текст.
- В DevTools → Network → Response Headers вижу: Content-Security-Policy: default-src 'self'; script-src 'self', X-Frame-Options: DENY, X-Content-Type-Options: nosniff.

---

## Дополнительные задания (на отлично)

### Доп. задание — Лёгкий уровень: «Управление ролями из админ-панели»

**Сценарий:** админ может менять роли пользователей прямо из панели — повышать до Author или понижать до User.

### Требования:

- На странице Admin Panel рядом с каждым пользователем — выпадающий список ролей и кнопка «Применить».
- При смене роли отправляется POST-запрос (через форму с антифоггери-токеном, не AJAX — классическая форма).

- Сервер через `UserManager.RemoveFromRolesAsync` очищает старые роли и `UserManager.AddToRoleAsync` назначает новую.
- Нельзя снять роль `Admin` с самого себя (защита от случайной блокировки).
- После смены роли — пользователь при следующем запросе получает обновлённый cookie с новой ролью. Проверьте: если пользователь был онлайн и его повысили до `Author` — он сможет зайти на страницу, доступную только `Author`, без перелога. Identity обновляет claims в cookie автоматически при следующем запросе (через `SecurityStampValidator`).

#### Контрольная точка:

- Админ меняет роль пользователя `B` с `User` на `Author` → `B` может открыть страницу, доступную только `Author`.
  - Админ пытается снять роль `Admin` с себя → ошибка, действие запрещено.
  - Обычный пользователь пытается открыть админ-панель → 403.
- 

#### Доп. задание — Средний уровень: «Двухфакторная аутентификация (2FA)»

**Сценарий:** пользователь может включить 2FA через приложение-аутентификатор (Google Authenticator, Microsoft Authenticator). При входе — дополнительный ввод кода.

#### Требования:

- На странице профиля — кнопка «Включить 2FA». При нажатии:
  - Сервер генерирует уникальный ключ через `UserManager.GenerateTwoFactorTokenAsync` (или через `AuthenticatorTokenProvider`).
  - Формируется QR-код (можно через библиотеку `QRCode` или через `URL otpauth://totp/...`).
  - Пользователь сканирует QR-код приложением на телефоне.
  - Пользователь вводит 6-значный код из приложения → сервер проверяет через `UserManager.VerifyTwoFactorTokenAsync` → 2FA включена.
- При входе (после ввода логина и пароля):
  - Если 2FA включена — пользователь попадает на страницу ввода 6-значного кода.
  - Код проверяется через `SignInManager.TwoFactorSignInAsync`.
  - Если код верный — вход завершён. Если неверный — ошибка.
- Кнопка «Отключить 2FA» — сбрасывает настройку.
- После включения 2FA при входе без кода — доступ не предоставляется, даже если пароль верный.

#### Контрольная точка:

- Включаю 2FA → сканирую QR-код → ввожу код → 2FA активна.
  - Выхожу → вхожу с правильным паролем → попадаю на страницу ввода кода.
  - Ввожу код из приложения → вхожу.
  - Ввожу неверный код → ошибка, остаюсь на странице ввода.
  - Отключаю 2FA → вхожу без кода → сразу попадаю на главную.
- 

#### Доп. задание — Сложный уровень: «Refresh-токены и отзыв JWT»

**Сценарий:** JWT живёт 15 минут, а refresh-токен позволяет получить новый JWT без повторного ввода пароля. Refresh-токен можно отозвать (разлогинить пользователя принудительно).

#### Требования:

- При логине через API сервер выдаёт ДВА токена:
  - **Access token** (JWT) — срок жизни 15 минут, содержит claims.
  - **Refresh token** — случайная строка (GUID), сохраняется в базе в таблице `RefreshTokens` (поля: `Id`, `Token`, `UserId`, `ExpiresAt`, `RevokedAt`, `CreatedAt`). Срок жизни — 7 дней.

- Клиент использует access token для запросов. Когда access token истёк — клиент отправляет refresh token на POST /api/auth/refresh и получает новую пару токенов.
- Refresh token отзывается (поле RevokedAt заполняется) при:
  - Выходе (POST /api/auth/logout — помечает refresh token как отозванный).
  - Получении нового токена через refresh (старый refresh отзывается, выдаётся новый — ротация).
- Если клиент отправляет отозванный refresh token → 401, нужно логиниться заново.
- Реализуйте эндпоинт POST /api/auth/logout-all — отзывает ВСЕ refresh-токены пользователя. Полезно при подозрении на компрометацию: «разлогинить на всех устройствах».
- На стороне клиента (можно простым JS в консоли браузера) реализуйте логику: при 401 на любой запрос — попробовать обновить токен через refresh, если refresh тоже не работает — редирект на логин.

#### **Контрольная точка:**

- Логинюсь → получаю access + refresh.
- Работаю 15 минут → access истёк → отправляю refresh → получаю новую пару.
- Старый refresh после ротации больше не работает → 401.
- Нажимаю «Выйти на всех устройствах» → все refresh-токены отозваны → на других устройствах нельзя обновить access.
- Проверьте: access token с истёкшим сроком отклоняется, даже если подпись верная.

#### **Пояснение:**

Главная проблема JWT — его нельзя отозвать. Он живёт до exp и всё. Refresh-токены решают это: access-токен делают коротким (15 минут), а отзыва управляют через refresh-токен, который хранится в базе. Если нужно «выгнать» пользователя — отзываем refresh, и через 15 минут его access-токен истечёт сам. Пользователь не сможет получить новый — refresh отозван.

---